Data Structures and Algorithms — Assignment 4

Campus Bike-Share: Data Structures End to End

Total: 100 points

Scenario

The university is launching a campus bike-share. The system tracks **40,000 bikes** across **220 docking stations**. Each bike has a unique integer `bikeID`. Each station has an `(x, y)` position in metres on a campus grid and a walking-distance graph connecting it to nearby stations.

Every question below is about the structures and algorithms behind that system. You are not building the product; you are making and defending the engineering choices inside it.

What to submit

One notebook (`.ipynb`) or script (`.py`) containing your code, together with your written answers. Written answers may go in markdown cells or in a separate PDF — but every question must be answered somewhere, clearly labelled.

- Code is graded on **whether it works and whether it meets the stated complexity**, not on style, naming or formatting.
- Where a question asks you to justify, derive or compare, **the reasoning is what earns the marks**. Several answers are defensible; an unsupported assertion is not one of them.
- If you use an external source, cite it.

Q1. Choosing the right structure (12 points)

The system needs three operations on its 40,000 bikes:

- `lookup(bikeID)` — called on every dock and undock, thousands of times an hour.
- `insert(bikeID)` / `remove(bikeID)` — a bike enters or leaves service, a few times a day.
- `report()` — list every bike in ascending `bikeID` order, once nightly.

(a) [5 pts] Choose one data structure for this workload and justify the choice against the *access pattern*, not in the abstract.

(b) [4 pts] The nightly report becomes an **hourly** report. Does your answer change? Argue either way, but argue from the cost of the operations.

(c) [3 pts] State the average-case time complexity of all four operations under your chosen structure.

Q2. Hashing (14 points)

(a) [4 pts] Explain the difference between **separate chaining** and **open addressing**, and give one concrete advantage of each over the other.

(b) [3 pts] A hash table has 16 slots and holds 12 entries. State the load factor alpha. Explain what happens to the expected probe length under **linear probing** as alpha approaches 1, and why.

(c) [3 pts] Why is deletion harder under open addressing than under separate chaining? Describe the standard fix and what it costs.

(d) [4 pts] A table has 7 slots, indices 0–6, and uses `h(k) = k mod 7` with **linear probing**. Insert the following keys **in this order**:

```
15, 11, 27, 8, 22
```

Show the final contents of the table, slot by slot. Show your probing for any key that collides.

Q3. The k nearest stations (18 points)

A rider at position `(ux, uy)` asks for the `k` nearest stations.

(a) [10 pts] Implement:

```
def k_closest_stations(stations, user, k):
    """stations: list of objects with .id, .x, .y
       user:     tuple (ux, uy)
       k:        how many to return
       returns:  list of station ids, nearest first"""
```

Your solution must run in **O(n log k)** time, where `n = len(stations)`. Handle `k <= 0` and `k > n` sensibly.

(b) [4 pts] Justify the complexity of your implementation. Say where each factor comes from.

(c) [4 pts] For `n = 40,000` and `k = 5`, explain why a bounded heap beats sorting the whole list, and identify the value of `k` at which that advantage disappears.

Q4. Shortest walking route (20 points)

Stations form a weighted undirected graph; edge weights are walking times in seconds.

(a) [10 pts] Implement Dijkstra's algorithm:

```
def shortest_route(graph, source, target):
    """graph:  dict mapping station id -> list of (neighbour_id, weight)
       returns: (total_seconds, [source, ..., target]) or (inf, []) if
                unreachable"""
```

You must return the **path**, not only the distance.

(b) [6 pts] Derive the time complexity of your implementation using a binary heap. Show where each term comes from in terms of `V` and `E` — do not simply state the final result.

(c) [4 pts] Dijkstra's algorithm is not correct on graphs with negative edge weights. Give a **concrete counterexample with three nodes**: list the edges and weights, state what Dijkstra returns, state the true shortest distance, and explain in one sentence which assumption the algorithm violated.

Q5. Amortized analysis (12 points)

The bike log is a dynamic array that starts with capacity 1 and **doubles** whenever it is full, copying all existing elements into the new allocation.

(a) [6 pts] Using the **aggregate method**, show that `append` costs amortized O(1). Show the total copy cost across `n` appends as a sum, and evaluate it.

(b) [6 pts] A colleague proposes growing by a **fixed +10 slots** instead of doubling, arguing it "wastes less memory." Derive the total copy cost across `n` appends under this scheme, state the resulting amortized cost per append, and say whether the proposal is a good trade.

Q6. Debugging (12 points)

The following function is supposed to find `target` in an ascending, duplicate-free list of station ids and return its index, or `-1` if absent. It is **wrong in two distinct ways**.

```
def find_station(sorted_ids, target):
    lo, hi = 0, len(sorted_ids)
    while lo <= hi:
        mid = (lo + hi) // 2
        if sorted_ids[mid] == target:
            return mid
        elif sorted_ids[mid] < target:
            lo = mid
        else:
            hi = mid - 1
    return -1
```

(a) [6 pts] Identify **both** faults. For each, name the line and explain what goes wrong.

(b) [4 pts] One of the two faults **cannot be observed while the other is present** — no input will ever expose it. Say which fault is hidden and why, give a concrete input (the list and the target) that demonstrates the failure you *can* observe, and give the input that would expose the hidden fault once the first one is repaired.

(c) [2 pts] Give the corrected function.

Q7. Designing a spatial query (12 points)

The system must answer: **"which bikes are within 2 km of this point?"** — up to 500 times a second, over 40,000 bikes whose positions change constantly.

Compare **two** approaches. One must be the naive linear scan; the second is yours to choose (a uniform grid, a quadtree, a k-d tree, geohashing, or another structure you can defend).

For each approach address:

- query time complexity, and what it depends on;
- the cost of a bike **moving** (position updates are constant);
- memory overhead.

Then **recommend one** and justify the recommendation against this specific workload — 500 queries/second, 40,000 moving objects, a 2 km radius on a campus roughly 5 km across.

There is no single correct recommendation. A well-argued case for either approach earns full credit; an unsupported preference does not.